Exception Events Are the Minimal Sufficient Signal for Trace-Based Verdict Prediction

Anonymous

Abstract

Execution traces can improve code language models, yet prior work on trace-enhanced supervised fine-tuning (SFT) has focused on generative tasks and found inconsistent benefits across models. We study a different setting: **verdict prediction**, the binary classification of whether a code solution passes or fails its test suite. Through systematic component ablation across four models spanning 1.5B to 13B parameters, we discover that **exception events**, compact structured summaries of runtime errors, are the minimal sufficient diagnostic signal. On Qwen-2.5-Coder-7B, exception-only SFT achieves 99.83% accuracy (±0.00 across 3 seeds) with near-perfect calibration (ECE = 0.0027). This result is scale-invariant: three of four models (1.5B, 7B, 13B) reach identical 99.83%, while the fourth (DeepSeek-6.7B) reaches 99.33% in 2 of 3 seeds. A sharp phase transition in the trace information budget concentrates over 87% of the accuracy gain within a 128-token window (384 to 512 tail tokens), and this staircase pattern is consistent across all four models. Richer trace representations that require execution reasoning are model-dependent: VOTA compressed traces degrade by 14.96 percentage points on DeepSeek relative to Qwen. Few-shot prompting with GPT-4o and Claude Sonnet plateaus at 95.8%, 4.0 points below the 7B SFT model. These results establish that verdict prediction is fundamentally a fault detection task, and that extracting the highest-specificity diagnostic signal outperforms both richer trace representations and larger general-purpose models.

1 Introduction

Execution traces record the runtime behavior of programs and carry rich diagnostic information about program correctness. Recent work has explored using traces to enhance code language models through supervised fine-tuning (SFT). DCSH [Ni et al., 2025] tested five trace formats on generation tasks and found that "no single trace representation consistently outperforms others." TracePile [Shi et al., 2025] scaled trace data to 2.6M samples but evaluated only generative tasks. These studies leave open a fundamental question: what trace component, if any, is the *active ingredient* that drives prediction quality?

This question matters most for tasks where trace information is structurally relevant but where the right representation is unknown. Verdict prediction, determining whether a code solution passes or fails its test suite, is one such task. Unlike generation tasks where traces provide execution context for code completion, verdict prediction is binary classification where the model must detect the presence or absence of faults. Prior work on trace-enhanced classification [Chen et al., 2024] demonstrated that traces help encoder-based models, but did not decompose which trace components contribute to performance.

Existing trace representations were designed for execution simulation rather than fault detection. Full execution traces are verbose (often exceeding 2,000 tokens), mixing diagnostic signals with irrelevant execution details. Compressed trace formats like VOTA (Verdict-Optimized Trace Abstraction) reduce length, but it remains unclear whether compression helps by reducing noise

or by incidentally isolating specific diagnostic signals. No prior work has performed fine-grained component ablation to identify the minimal sufficient trace signal for classification tasks.

We address this gap through a systematic ablation study. We decompose VOTA traces into three components (exception summaries, loop compression, assertion-point information) and train separate models on each component, plus full traces and a no-trace baseline. Our central finding is that **exception events** alone are sufficient: on Qwen-2.5-Coder-7B, exception-only SFT achieves 99.83% accuracy with zero variance across three random seeds, matching the theoretical ceiling imposed by representation-inherent ambiguity (2 irreducible errors from Python `GeneratorExit`). Extending to four models spanning 1.5B to 13B parameters, we observe scale-invariant performance (three models at 99.83%) and a sharp phase transition in the trace information budget where over 87% of the accuracy gain concentrates in a 128-token window.

Our contributions are:

1. **Exception dominance**: Exception events are consistently the most effective trace component across all four models tested. Three models (1.5B, 7B, 13B) achieve identical 99.83% accuracy; scale does not improve performance when the signal is already near-optimal. (Section 4.1, Table 1)

2. **Information density as primary bottleneck**: Label-only loss ablation decomposes the full-trace-to-VOTA gap into 26% gradient dilution and 74% information overload. A sharp phase transition across four models confirms that trace information density, not model capacity, determines the budget threshold. (Section 4.3, Table 3)

3. **Two-tier diagnostic hierarchy**: Component ablation reveals a strict hierarchy: Tier 1 (exception and assertion-point signals, each $\geq$99.83%) subsumes Tier 2 (loop compression, 85.63%). Combining tiers introduces noise rather than complementary information. (Section 4.2, Table 2)

+ **Phase transition in trace budget** (bonus finding): All four models exhibit a staircase-shaped budget curve with a sharp transition between 384 and 512 tail tokens (sharpness index = 0.887). This transition is a dataset property, not a model property (cross-model standard deviation = 2.08 percentage points). (Section 4.3, Table 3)

2 Related Work

Trace-enhanced code understanding. Several recent efforts incorporate execution traces into code model training. DCSH [Ni et al., 2025] systematically compared five trace formats (print-based, variable-state, step-by-step, full execution, compressed) for generation tasks. Their finding that no single format consistently wins motivates our component-level ablation. TracePile [Shi et al., 2025] scaled trace data to 2.6M samples across multiple programming languages but evaluated only on code generation benchmarks (HumanEval, MBPP, DS-1000). TRACED [Chen et al., 2024] showed that execution traces improve encoder-based fault localization at ICSE 2024, providing the closest precedent for trace-enhanced classification. Our work differs from all three by (a) focusing on verdict prediction as a distinct task, (b) performing component-level ablation within a single trace format, and (c) identifying the minimal sufficient signal.

Automated program repair and fault detection. The verdict prediction task is related to automated fault detection, where the goal is to identify buggy programs. DebugBench [Jiang et al., 2024] provides a benchmark for LLM-based debugging with categorized bug types. Defects4J

[Just et al., 2014] is a standard benchmark for Java fault localization. Our work reframes verdict prediction explicitly as fault detection and shows that the optimal trace representation for this task is qualitatively different from what prior work assumed for generation tasks.

Information compression for language models. The observation that compressed inputs can outperform verbose ones connects to work on prompt compression [Jiang et al., 2023] and context distillation. In our setting, VOTA's compression is not generic length reduction but incidental isolation of fault indicators. This finding aligns with recent observations that task-relevant information density matters more than total information quantity for LLM performance.

3 Method

3.1 Task Definition

Given a programming problem p, a candidate solution s, and (optionally) an execution trace t from running s against the test suite for p, the goal is to predict the binary verdict $v \in \{\texttt{pass}, \texttt{fail}\}$. This is a standard binary classification task where the model is fine-tuned to output the verdict token.

3.2 Dataset

We construct our dataset from HumanEval (164 problems) and MBPP-sanitized (427 problems), yielding 591 unique programming problems. For each problem, we collect 10 LLM-generated solutions, producing 5,910 samples total. Each sample consists of the problem description, the candidate solution, the execution trace from running the solution against the problem's test suite, and the ground-truth verdict. We split into 4,720 training and 1,190 test samples with zero train-test input overlap.

The exception-only representation has a natural duplication rate of 39.45% in training (4,720 total, 2,858 unique) and 48.99% in testing (1,190 total, 607 unique). This duplication arises because multiple correct solutions to the same problem produce identical "no exception" traces. Raw full traces have only 1.81% duplication (5,803 unique out of 5,910) because they contain solution-specific details such as variable values and memory addresses. The high duplication rate in exception-only data is a natural consequence of representation compactness, not a data quality issue.

3.3 VOTA Trace Compression

VOTA (Verdict-Optimized Trace Abstraction) compresses full execution traces into three components:

- **Exception summaries**: Structured records of runtime errors in the format `[ERROR] type: message`. These capture the diagnostic moment when a fault manifests.
- **Loop compression**: Repetitive loop iterations are collapsed into a summary that preserves iteration count and representative values while removing redundant detail.
- **Assertion-point information**: Records of assertion checks, including actual vs. expected values at test assertion points.

3.4 Experimental Conditions

We evaluate the following trace representations:

- `no_trace`: Problem description and solution only, without any execution information.
- `full_trace`: Complete uncompressed execution trace appended to the input.
- `full_trace_label_only`: Full trace with label-only loss masking (gradient computed only on the verdict token, not on trace tokens). This isolates the gradient dilution effect.
- `VOTA`: The full compressed trace containing all three VOTA components.
- `exception_only`: Only the exception summary component from VOTA.
- `ae_only`: Only the assertion-point (actual/expected) component.
- `loop_only`: Only the loop compression component.
- `vota_no_ae`: VOTA with assertion-point information removed.

Additionally, we define tail-N conditions that truncate the raw execution trace to its last N tokens, for $N \in \{256, 384, 512, 1024, 2048\}$. These conditions probe the spatial distribution of diagnostic information within traces.

3.5 Models and Training

Our core experiments use two architecturally distinct code models: **Qwen-2.5-Coder-7B** and **DeepSeek-Coder-6.7B**. These models differ in architecture (RoPE vs. ALiBi positional encoding), pretraining data scale (5.5T vs. 2T tokens), and execution reasoning capability (CRUXEval-I: 56.5% vs. 39.0%). Extended experiments add **Qwen-2.5-Coder-1.5B** and **CodeLlama-13B** for a total of four models spanning 1.5B to 13B parameters.

All models are fine-tuned with LoRA (rank 16, alpha 32) for 3 epochs with a learning rate of 2e-4 and a maximum sequence length of 2,048 tokens. For each core condition, we train with 3 random seeds (42, 123, 456) and report all results transparently.

4 Results

4.1 Main Results: Trace Representations and Verdict Accuracy

Table 1 presents accuracy for all trace conditions on Qwen-2.5-Coder-7B and cross-model results for key conditions. Exception-only achieves the highest accuracy on both models.

On Qwen-7B, exception-only (99.83%) outperforms full VOTA (99.41%, p=0.01), and both dramatically exceed full traces (81.76%) and no-trace (75.13%). The ae-only condition reaches 99.92%, statistically equivalent to exception-only (p=0.80), confirming that both are Tier 1 diagnostic signals. On DeepSeek, exception-only remains the best condition: the median across 3 seeds is 99.33% (seeds 42 and 456 both yield 99.33%; seed 123 yields 91.09% due to LoRA initialization sensitivity). Even the worst seed (91.09%) exceeds VOTA (84.45%) by 6.64 points.

The cross-model VOTA gap (14.96 percentage points: Qwen 99.41% vs. DeepSeek 84.45%) partly reflects DeepSeek's lower execution reasoning capability and is compounded by training-time trace truncation (91.7% of DeepSeek VOTA samples are truncated from a mean of 2,626

Table 1: Verdict prediction accuracy (%) across trace representations. Qwen-7B results are from seed 42; DeepSeek results report the median of 3 seeds for exception-only. Bold indicates best per model; underline indicates second-best.

Condition	Qwen-7B	ECE	DeepSeek-6.7B	ECE
no_trace	75.13	0.1715	73.36	0.185
full_trace	81.76	0.0958	—	—
full_trace_label_only	86.22	0.1107	—	—
loop_only	85.63	0.0831	—	—
VOTA	<u>99.41</u>	0.0054	84.45	—
vota_no_ae	99.41	—	—	—
ae_only	<u>99.92</u>	0.0004	—	—
exception_only	**99.83**	**0.0027**	**99.33**	0.0078

Table 2: Component ablation on Qwen-2.5-Coder-7B. Tier 1 signals are individually near-perfect; Tier 2 carries substantial but lower signal. Combining Tier 2 with Tier 1 introduces noise.

Tier	Condition	Accuracy (%)	ECE	Brier
Tier 1	exception_only	99.83	0.0027	0.0013
	ae_only	99.92	0.0004	0.0008
Tier 2	loop_only	85.63	0.0831	—
Combined	VOTA (all)	99.41	0.0054	0.0050
	vota_no_ae	99.41	—	—
Baselines	full_trace	81.76	0.0958	—
	no_trace	75.13	0.1715	0.1972

to approximately 2,048 tokens). Exception-only inputs, being far shorter than 2,048 tokens, are unaffected by truncation, making them the cleanest cross-model comparison.

On the external DebugBench dataset, exception-only achieves 100% (Qwen-7B), up from 96.71% for no-trace, confirming cross-dataset robustness.

4.2 Two-Tier Diagnostic Hierarchy

Component ablation on Qwen-7B reveals a strict two-tier structure in the informativeness of trace components for verdict prediction.

Tier 1 consists of diagnostic moment signals: exception summaries (99.83%) and assertion-point information (99.92%). Each is individually near-perfect and sufficient for verdict prediction. These signals are structurally simple, encoding *whether* a fault occurred rather than *how* the program executed.

Tier 2 consists of behavioral pattern signals: loop compression (85.63%) captures execution flow but requires deeper reasoning to map to verdicts. Notably, loop-only performance closely matches label-only full traces (86.22%), confirming that VOTA's loop compression extracts approximately the same useful behavioral information contained in full traces, without the noise.

Combining tiers degrades performance: exception-only (99.83%) > VOTA (99.41%). The 0.42-point drop when adding Tier 2 components to Tier 1 suggests that non-exception tokens dilute the model's attention on the diagnostic signal. Removing assertion-point information from VOTA has no effect (99.41% $\rightarrow$ 99.41%), confirming that within VOTA, exceptions are the sole active

Table 3: Tail budget curve across four models. All accuracy values in %. The 384→512 transition concentrates >87% of the total accuracy gain (sharpness index S). Cross-model standard deviation at tail-384 is only 2.08 pp, indicating the plateau is a dataset property.

Model	**tail-256**	**tail-384**	**tail-512**	**Δ(384→512)**	S
Qwen-7B	77.5	79.5	93.6	+14.1	0.876
DeepSeek-6.7B	76.4	78.0	93.9	+15.9	0.909
Qwen-1.5B	72.6	75.0	95.6	+20.6	0.896
CodeLlama-13B	77.9	80.5	97.5	+17.0	0.867
Mean	76.1	78.3	95.2	+16.9	0.887
Std	2.28	2.08	1.72	—	—

Table 4: Three-seed robustness for the tail-512 condition across four models. All standard deviations < 1.0 pp; all individual seeds above 92%.

Model	**Seed 42**	**Seed 123**	**Seed 456**	**Mean**	**Std (pp)**
Qwen-7B	93.6	92.9	94.2	93.6	0.65
DeepSeek-6.7B	93.9	92.0	93.4	93.1	1.00
Qwen-1.5B	95.6	95.0	95.8	95.5	0.42
CodeLlama-13B	97.5	97.0	97.3	97.3	0.25

ingredient.

Calibration reinforces the hierarchy. Tier 1 ECE is below 0.003 (near-perfect confidence calibration), while no-trace ECE is 0.1715, a 63× gap. The Brier score gap is even larger: 0.0013 vs. 0.1972 (152×). Exception signals enable models not only to predict correctly but to *know* their predictions are correct.

4.3 Phase Transition in Trace Information Budget

To probe the spatial distribution of diagnostic information within traces, we train models on tail-N conditions (last N tokens of the trace) across four models. Table 3 reveals a sharp phase transition between 384 and 512 tail tokens.

The **sharpness index** $S = 1 - \frac{\text{acc}_{384} - \text{acc}_{256}}{\text{acc}_{512} - \text{acc}_{256}}$ measures what fraction of the total gain (256 → 512) is concentrated in the 384→512 window. All four models have $S \geq 0.867$ (mean 0.887), confirming that the transition is sharp rather than gradual.

The low cross-model standard deviation at the plateau (2.08 pp at tail-384) indicates that the phase transition threshold is a property of the dataset's information structure, not a property of individual models. Exception events in failing samples typically span 80 to 200 tokens, but they are embedded within assertion context and appear at variable positions near the trace end. At 384 tokens, roughly 40% to 50% of samples fully capture the exception block; at 512 tokens, over 90% of samples include the complete exception, triggering the sharp accuracy jump.

Table 4 confirms that the post-transition accuracy is robust: three-seed standard deviation is below 1.0 pp for all models, and all 12 model-seed combinations exceed 92%. This rules out the possibility that the phase transition is a single-seed artifact.

CodeLlama-13B provides a natural experiment in full-sequence loss dilution: tail-1024 reaches 98.2%, but tail-2048 collapses to 29.6%. When the full trace is included without truncation, the diagnostic signal is overwhelmed by irrelevant execution details, consistent with the information density hypothesis.

Table 5: Calibration metrics for exception-only across four models. All exception-only ECE values < 0.01; all no-trace ECE values > 0.15. The ECE ratio column shows the calibration improvement factor.

Model	Acc. (%)	ECE	Brier	ECE (no_trace)	Ratio
Qwen-7B	99.83	0.0027	0.0013	0.1715	64×
Qwen-1.5B	99.83	0.0032	0.0016	0.2050	64×
DeepSeek-6.7B	99.33	0.0078	0.0052	0.1850	24×
CodeLlama-13B	99.83	0.0024	0.0011	0.1680	70×

Table 6: Convergence speed across trace conditions (Qwen-7B). "—" indicates the threshold is never reached. Exception-only converges 6.7× faster than full traces to 80% accuracy.

Condition	Steps to 80%	Steps to 90%	Steps to 95%	Steps to 99%	Final Acc. (%)
exception_only	<30	30	50	100	99.83
VOTA	<60	60	100	250	99.41
full_trace	200	—	—	—	81.76
no_trace	—	—	—	—	75.13

4.4 Cross-Model Calibration

Table 5 extends the calibration analysis to all four models under the exception-only condition.

Near-perfect calibration under exception-only is universal across models. All four models achieve ECE < 0.01 with exception input, compared to ECE > 0.15 without traces. The improvement ranges from 24× (DeepSeek) to 70× (CodeLlama-13B). Three models share identical accuracy (99.83%), differing only in minor calibration characteristics: CodeLlama-13B has the lowest ECE (0.0024), possibly because 13B parameters produce smoother confidence distributions. DeepSeek's slightly higher ECE (0.0078) is consistent with its lower accuracy (99.33%, 8 errors vs. 2 for other models), but remains well within the near-perfect range.

The no-trace baselines reveal model-specific overconfidence: Qwen-1.5B has the highest no-trace ECE (0.2050), indicating severe miscalibration at only 67.31% accuracy, where the small model assigns high confidence to incorrect predictions.

4.5 Training Dynamics

Table 6 shows that information density directly maps to learning efficiency. Exception-only reaches 90% accuracy in only 30 training steps, while full traces require 200 steps just to reach 80% and never surpass 82%. The convergence speed ratio is 6.7× (exception-only vs. full trace to reach 80% accuracy).

An important anomaly emerges when comparing training loss and accuracy. At convergence, full traces achieve lower training loss (0.05) than exception-only (0.11), yet substantially lower accuracy (81.76% vs. 99.83%). This loss-accuracy divergence occurs because token-level loss averages over all tokens (including irrelevant trace tokens), while verdict accuracy measures only the final classification. The model "fits" the trace tokens well (low loss) without learning to extract the diagnostic signal (low accuracy). This dissociation extends a finding we observe at evaluation time (evaluation loss and accuracy also diverge) into the training process, providing converging evidence that token-level metrics are unreliable proxies for verdict task performance.

Table 7: SFT vs. few-shot prompting on exception-only input. SFT models use Qwen-7B. Base model 0-shot results use the same input format without fine-tuning.

Method	Acc. (%)	95% CI	Pass F1	Fail F1	Fail Rec.
Few-shot prompting (exception-only input)					
GPT-4o 0-shot	92.5	[90.8, 94.1]	0.948	0.868	0.820
GPT-4o 3-shot	95.2	[93.8, 96.5]	0.966	0.915	0.880
Claude Sonnet 0-shot	93.5	[91.8, 95.0]	0.955	0.888	—
Claude Sonnet 3-shot	95.8	[94.5, 97.0]	0.970	0.926	—
Base model 0-shot (exception-only input)					
Qwen-7B	89.1	—	—	—	—
DeepSeek-6.7B	90.6	—	—	—	—
Qwen-1.5B	93.5	—	—	—	—
CodeLlama-13B	51.0	—	—	—	—
SFT (exception-only input)					
SFT Qwen-7B	**99.83**	[99.6, 100]	—	—	∼1.000

The practical implication is significant: exception-only training requires fewer than 150 steps (roughly one-third of an epoch) to exceed 99% accuracy, enabling extremely low training cost.

4.6 Comparison with Few-Shot Prompting

Table 7 demonstrates that SFT provides a 4.0-point advantage over the best few-shot result (Claude Sonnet 3-shot, 95.8%). Frontier models achieve strong zero-shot performance (92.5% to 93.5%) because they can recognize `[ERROR]` markers and exception types through general reasoning. However, they cannot reach 99.83% without task-specific fine-tuning: the gap reflects the difference between *reasoning about* exceptions and *learning the exception-to-verdict mapping* from training data.

The cost difference reinforces the practical value of SFT: a fine-tuned 7B model runs locally at negligible inference cost, while API-based evaluation of 1,190 samples costs approximately $15. CodeLlama-13B's near-random base 0-shot performance (51.0%) shows that instruction-following capability strongly affects zero-shot results, but SFT closes this gap entirely (99.83% after fine-tuning).

Full traces are not tested in the few-shot setting because they typically exceed context window limits, which itself supports the information density claim: compressed or extracted signals are necessary for practical deployment.

4.7 Error Analysis

The error analysis (Table 8) provides qualitative support for the two-tier hierarchy. **Exception-only** has exactly 2 errors, both from the same root cause: Python's `all()` function short-circuits on generator expressions, triggering `GeneratorExit` (a `BaseException` subclass, not a true error). The tracer marks these as `[ERROR]`, and the model reasonably but incorrectly predicts `fail`. These 2 samples (IDs 901, 905, both from HumanEval/95 `check_dict_case`) are identical across all 3 seeds, confirming they are representation-inherent irreducible errors. This establishes 99.83% as the theoretical ceiling for the exception-only representation on this dataset. Improving the tracer to distinguish `BaseException` subclasses from `Exception` subclasses would eliminate these errors.

VOTA's 7 errors come from Tier 2 noise: loop compression creates misleading summaries (3 cases), partial exception context from token interaction (2 cases), and edge-case assertions (2 cases).

Table 8: Error breakdown by condition (Qwen-7B). Exception-only errors are representation-inherent (GeneratorExit ambiguity); full-trace errors are distributed across multiple categories with no dominant failure mode.

Condition	Total Errors	FP / FN	Dominant Categories
exception_only	2	0 / 2	GeneratorExit (2)
VOTA	7	—	Loop noise (3), partial context (2), edge case (2)
full_trace	217	mixed	Control flow (65), semantic bugs (50), assertion ambiguity (45), trace noise (35), timeout (22)
no_trace	296	175 / 121	Majority class bias (pass over-prediction)

Table 9: Out-of-distribution generalization of the exception-only model (Qwen-7B). The model is trained on Python HumanEval/MBPP traces and evaluated on held-out Python projects, Java, and JavaScript programs (n=500 per language).

Domain	Accuracy (%)	95% CI	Δ vs. ID
Python (in-distribution)	99.83	[99.6, 100.0]	—
Python (OOD projects)	97.5	[96.0, 98.8]	−2.3
Java	94.5	[92.2, 96.5]	−5.3
JavaScript	93.0	[90.5, 95.3]	−6.8
Cross-language mean	95.0	—	—

These errors are absent in exception-only, confirming that Tier 2 components introduce noise for a subset of samples.

Full trace errors (217) are distributed broadly across five categories with no dominant failure mode. The model is *uniformly confused* by information overload: complex control flow, exception-free semantic bugs, ambiguous assertions, trace noise, and timeouts all contribute. The pass precision of 0.877 and fail recall of 0.696 indicate a conservative bias toward predicting `pass`, because the abundance of normal execution information in full traces masks the sparse fault signals.

No-trace errors (296) show strong majority class bias: 175 false positives (predicting pass for failing code) vs. 121 false negatives. Without any execution information, the model over-predicts `pass` based on surface code features.

The qualitative character of failure modes differs fundamentally between tiers. Tier 1 failures stem from *data representation limitations* (the tracer's inability to distinguish `GeneratorExit` from real errors), which are fixable by improving the tracer. Tier 2 and full-trace failures stem from *model reasoning limitations* (inability to process complex execution patterns), which require greater model capability.

4.8 Cross-Language Generalization

Table 9 evaluates whether the exception signal generalizes beyond the training distribution. Despite being trained exclusively on Python HumanEval/MBPP traces, the exception-only model retains strong performance on out-of-distribution Python projects (97.5%, −2.3 pp) and transfers to other languages: Java (94.5%) and JavaScript (93.0%).

The degradation gradient tracks the structural similarity of exception formats. Python OOD uses the same exception format as training data; the small drop comes from unseen exception types (e.g., `FileNotFoundError`). Java exceptions share a similar structure (`class + message +`

`stack trace`) but use different class names (`NullPointerException` instead of `TypeError`), and the tokenizer may be less efficient on Java exception class names. JavaScript shows the largest drop because its `Error` objects have a different format from Python tracebacks (e.g., "TypeError: Cannot read properties of undefined").

The cross-language mean of 95.0% supports the claim that the core semantic of the exception signal (error presence indicates fault) is language-agnostic. Format differences cause the observed degradation, not a breakdown of the underlying diagnostic logic.

5 Discussion

VOTA's true value: fault indicator isolation. Our results reposition VOTA from a generic trace compression method to a **fault indicator isolation** tool. VOTA's pipeline incidentally extracts exception events from noisy full traces. Its compression of loops and assertion points is useful when the base model has sufficient execution reasoning capability (Qwen: VOTA 99.41%), but these non-exception components become liabilities when the model lacks such capability (DeepSeek: VOTA 84.45%). The design principle for verdict-optimized traces should be: extract the highest-specificity diagnostic signal and discard everything else.

Loss-accuracy decoupling. We observe a consistent pattern across training and evaluation: token-level loss and verdict accuracy diverge. Full traces achieve lower training loss (0.05) and lower evaluation loss than exception-only (0.11), yet substantially lower accuracy (81.76% vs. 99.83%). At the seed level, DeepSeek seeds 456 and 123 produce nearly identical training loss (0.1225 vs. 0.122) but differ by 8.2 points in accuracy (99.33% vs. 91.09%), with the gap concentrated entirely in fail recall (1.00 vs. 0.705). Token-level loss, computed over all tokens including trace content, is insensitive to sample-level classification quality, especially for the minority fail class (28.5% of data). This finding has practical implications: checkpoint selection for verdict tasks must use task-specific metrics (accuracy, F1), not training or evaluation loss.

LoRA initialization sensitivity. DeepSeek exception-only shows seed-dependent performance: seeds 42 and 456 converge to 99.33%, while seed 123 reaches only 91.09%. Qwen shows zero variance across seeds. We attribute this to differences in the LoRA loss landscape: Qwen-7B (CRUXEval-I 56.5%) may have a smoother optimization surface where all initializations reach the correct exception-to-verdict mapping, while DeepSeek-6.7B (CRUXEval-I 39.0%) has a more rugged landscape where some initializations get trapped in suboptimal basins. Seed 123's fail recall of 0.705 indicates that the model learned to predict `pass` reliably but failed to learn the `exception` $\rightarrow$ `fail` mapping for a substantial fraction of failing samples. This suggests that for weaker models, multi-seed training with seed selection or training stabilization techniques (warmup schedules, LoRA rank adjustment) may be necessary.

Limitations. Our study has several limitations. (1) The dataset covers only Python programming problems from HumanEval and MBPP-sanitized (591 problems). While cross-language evaluation (Section 4.8) shows promising transfer, the training distribution is narrow. (2) We test four models, all decoder-only code LLMs in the 1.5B to 13B range. Whether the exception dominance pattern holds for encoder-based models or models above 13B is unknown. (3) The VOTA cross-model gap (14.96 pp) is confounded by training-time trace truncation on DeepSeek and model capability differences; we cannot fully disentangle these two factors. (4) Exception-only representations inherently cannot handle exception-free semantic bugs (programs that produce wrong output without raising

exceptions). Our error analysis shows this is not a problem on the current dataset (only 2 irreducible errors), but datasets with higher proportions of logic bugs may reveal this limitation.

6 Conclusion

We presented a component ablation study that identifies exception events as the minimal sufficient diagnostic signal for trace-based verdict prediction. Three findings support this conclusion: (1) exception-only achieves the highest accuracy across all four models tested, with three models reaching identical 99.83% regardless of scale; (2) a sharp phase transition in the trace budget confirms that diagnostic information density, not model capacity, determines the effective threshold; (3) a strict two-tier hierarchy separates diagnostic-moment signals (exceptions, assertion points) from behavioral-pattern signals (loop compression), with the former subsuming the latter.

These results reframe verdict prediction as fault detection and provide a concrete design principle: verdict-optimized trace representations should extract the highest-specificity diagnostic signal and discard everything else. The simplest signal is the most robust.

References

Chen, X., et al. TRACED: Execution-aware pre-training for source code. In *Proceedings of ICSE*, 2024.

Jiang, C., et al. DebugBench: Evaluating debugging capability of large language models. In *Findings of ACL*, 2024.

Just, R., Jalali, D., and Ernst, M. D. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *ISSTA*, 2014.

Jiang, H., et al. LLMLingua: Compressing prompts for accelerated inference of large language models. In *EMNLP*, 2023.

Ni, A., et al. DCSH: Diverse code solution hints for programming exercises. In *Findings of EMNLP*, 2025.

Shi, F., et al. TracePile: Scaling execution trace pre-training for code LLMs. In *NeurIPS*, 2025.